

FLEET FINANCIAL MANAGEMENT SYSTEM

Complete Backend System Blueprint
AI-Ready Architecture Documentation — v1.0.0

Document Property	Value
System Name	Fleet Financial Management System (FFMS)
Backend Framework	Node.js + Express.js
ORM	Prisma
Database	PostgreSQL
Authentication	JWT (JSON Web Tokens) + bcryptjs
File Storage	Local filesystem (configurable to Cloudinary)
Version	1.0.0 — Initial Architecture Release
Document Status	Implementation-Ready Blueprint
Target Audience	AI Agents, Backend Engineers, Senior Developers

1. SYSTEM OVERVIEW

The Fleet Financial Management System (FFMS) is a web-based financial tracking and reporting platform built for a shipping company operating multiple vessels. It centralises income recording, expense tracking, payroll management, loan tracking, and financial reporting — all segmented per ship.

1.1 System Purpose

- Automate financial recording and eliminate manual entry errors
- Provide per-ship income, expense, and profit/loss tracking with date-range filtering
- Manage employee payroll and salary disbursement per vessel
- Track bank loans, monthly installments, and remaining balances
- Generate real-time financial reports (P&L, expense breakdown, income breakdown, loan summary)
- Enforce role-based access so only authorised users can write financial data

1.2 User Roles & Permissions

Role	Access Level	Full Capabilities
ADMIN	Full Access	All CRUD operations, user management, ship management, all reports, delete any record
ACCOUNTANT	Financial Access	Record income & expenses, manage loans, generate reports — CANNOT manage users or ships
MANAGER	Read-Only	View reports and dashboards ONLY — zero write access to any entity

1.3 Core Business Rules (Non-Negotiable)

- RULE-01: Every income and expense record MUST be linked to a ship_id. Records without a ship are not permitted.
- RULE-02: Profit = Total Income - Total Expenses. This is calculated per ship and company-wide using SQL aggregations.
- RULE-03: Expense categories are predefined ENUMs. No free-text category entry is allowed.
- RULE-04: Income types are predefined ENUMs: SALES or DELIVERY_ORDER only.
- RULE-05: When loan repayments are recorded, remaining_balance on loans table is atomically decremented via a Prisma \$transaction.
- RULE-06: When remaining_balance reaches zero, the loan status auto-updates to PAID_OFF.
- RULE-07: Salary expenses (category=SALARY) MUST reference a valid employee_id from the same ship.
- RULE-08: All API endpoints require a valid JWT Bearer token. No public endpoints except /auth/login.
- RULE-09: Ships cannot be hard-deleted if they have associated income or expense records.
- RULE-10: Users are soft-deleted only (is_active=false) — their historical created_by references must remain intact.

2. PROJECT STRUCTURE

The backend follows a modular architecture. Each business domain (ships, income, expenses, employees, loans, reports) is a self-contained module with its own routes, controller, service, and validation files.

```
ffms-backend/
├── prisma/
│   ├── schema.prisma      # All database models, ENUMs, and relations
│   ├── seed.js            # Seeds initial ADMIN user and 2 sample ships
│   └── migrations/        # Prisma auto-generated migration history
├── src/
│   ├── config/
│   │   ├── database.js    # Singleton Prisma client export
│   │   └── constants.js   # All ENUMs, status codes, defaults
│   ├── middlewares/
│   │   ├── auth.middleware.js # JWT token verification → sets req.user
│   │   ├── role.middleware.js # Role enforcement → authorize(...roles)
│   │   ├── validate.middleware.js # Joi schema validation → validate(schema)
│   │   ├── upload.middleware.js # Multer file upload handler
│   │   └── error.middleware.js # Global error handler (last middleware)
│   └── modules/
│       ├── auth/
│       │   └── auth.routes.js
```

```

├── auth.controller.js
├── auth.service.js
├── auth.validation.js
├── users/
│   ├── users.routes.js
│   ├── users.controller.js
│   ├── users.service.js
│   └── users.validation.js
├── ships/
│   ├── ships.routes.js
│   ├── ships.controller.js
│   ├── ships.service.js
│   └── ships.validation.js
├── income/
│   ├── income.routes.js
│   ├── income.controller.js
│   ├── income.service.js
│   └── income.validation.js
├── expenses/
│   ├── expenses.routes.js
│   ├── expenses.controller.js
│   ├── expenses.service.js
│   └── expenses.validation.js
├── employees/
│   ├── employees.routes.js
│   ├── employees.controller.js
│   ├── employees.service.js
│   └── employees.validation.js
├── loans/
│   ├── loans.routes.js
│   ├── loans.controller.js
│   ├── loans.service.js
│   └── loans.validation.js
├── reports/
│   ├── reports.routes.js
│   ├── reports.controller.js
│   └── reports.service.js
├── utils/
│   ├── asyncHandler.js      # Wraps async controllers: auto-calls next(err)
│   ├── apiResponse.js      # sendSuccess() and sendError() helpers
│   ├── pagination.js        # Extracts page/limit from query, returns skip/take
│   └── dateHelpers.js        # Parse date ranges, get month boundaries
├── routes/
│   └── index.js              # Mounts all module routers under /api/v1
├── app.js                    # Express app: global middleware + routes
├── uploads/                  # Receipt files stored here
├── .env                      # Environment secrets (git-ignored)
├── .env.example              # Template – committed to repo
├── .gitignore
├── package.json
└── server.js                 # Entry point: starts HTTP listener

```

3. DATABASE DESIGN

Database: PostgreSQL. ORM: Prisma. All primary keys are UUID type. All monetary values use DECIMAL(14,2) to avoid floating-point precision issues.

3.1 Table Relationships

Table	Relationships
users	No foreign keys. Standalone. Referenced by income_records.created_by, expense_records.created_by, loans.created_by
ships	Parent of: employees (1:many), income_records (1:many), expense_records (1:many)
employees	Belongs to: ships. Referenced by: expense_records.employee_id (salary expenses)
income_records	Belongs to: ships. Referenced by: users (created_by)
expense_records	Belongs to: ships. Optional FK to: employees.id (salary), loan_payments.id
loans	Parent of: loan_payments (1:many). No ship FK — company-wide liability
loan_payments	Belongs to: loans. Has optional reverse reference from expense_records

3.2 Table: users

Column	Type	Constraint	Description
id	UUID	PK, gen_random_uuid()	Unique identifier for the user
name	VARCHAR(100)	NOT NULL	Full display name
email	VARCHAR(150)	NOT NULL, UNIQUE	Login email — must be unique system-wide
password_hash	VARCHAR(255)	NOT NULL	bcrypt hash of user password (min 12 rounds)
role	UserRole ENUM	NOT NULL, DEFAULT ACCOUNTANT	Values: ADMIN ACCOUNTANT MANAGER
is_active	BOOLEAN	NOT NULL, DEFAULT true	Soft-disable flag — false blocks login
created_at	TIMESTAMP	DEFAULT NOW()	Auto-set on insert
updated_at	TIMESTAMP	AUTO UPDATE	Auto-updated on every change via Prisma

Indexes: *UNIQUE* on email. No composite indexes needed.

3.3 Table: ships

Column	Type	Constraint	Description
id	UUID	PK	Unique ship identifier
name	VARCHAR(150)	NOT NULL	Ship commercial or vessel name
registration_number	VARCHAR(100)	NOT NULL, UNIQUE	Official registration / IMO number

status	ShipStatus ENUM	NOT NULL, DEFAULT ACTIVE	Values: ACTIVE INACTIVE MAINTENANCE
description	TEXT	NULLABLE	Optional notes about the vessel
created_at	TIMESTAMP	DEFAULT NOW()	Auto-set on insert
updated_at	TIMESTAMP	AUTO UPDATE	Auto-updated on every change

3.4 Table: employees

Column	Type	Constraint	Description
id	UUID	PK	Unique employee identifier
ship_id	UUID	FK → ships.id, NOT NULL	The ship this employee is assigned to
name	VARCHAR(150)	NOT NULL	Employee full name
role	VARCHAR(100)	NOT NULL	Job title (e.g., Captain, Chief Engineer, Cook)
base_salary	DECIMAL(12,2)	NOT NULL, CHECK(>0)	Monthly base salary in company currency
is_active	BOOLEAN	DEFAULT true	Active flag — false = no longer on ship
created_at	TIMESTAMP	DEFAULT NOW()	Auto-set on insert
updated_at	TIMESTAMP	AUTO UPDATE	Auto-updated on every change

Indexes: INDEX on ship_id (frequently joined in expense queries for salary category).

3.5 Table: income_records

Column	Type	Constraint	Description
id	UUID	PK	Unique income entry identifier
ship_id	UUID	FK → ships.id, NOT NULL	Ship that earned this income
type	IncomeType ENUM	NOT NULL	Values: SALES DELIVERY_ORDER
amount	DECIMAL(14,2)	NOT NULL, CHECK(>0)	Income amount. Must be positive.
date	DATE	NOT NULL	Date the income was received/earned
description	TEXT	NULLABLE	Optional details (cargo type, client name, etc.)
created_by	UUID	FK → users.id	User who recorded this income
created_at	TIMESTAMP	DEFAULT NOW()	Auto-set on insert
updated_at	TIMESTAMP	AUTO UPDATE	Auto-updated on change

Indexes: COMPOSITE INDEX on (ship_id, date) — critical for P&L and filtered report queries.

3.6 Table: expense_records

Column	Type	Constraint	Description
id	UUID	PK	Unique expense entry identifier
ship_id	UUID	FK → ships.id, NOT NULL	Ship that incurred this expense
category	ExpenseCategory ENUM	NOT NULL	See full ENUM list below
amount	DECIMAL(14,2)	NOT NULL, CHECK(>0)	Expense amount. Must be positive.
date	DATE	NOT NULL	Date the expense was incurred
payment_method	PaymentMethod ENUM	NOT NULL	Values: CASH BANK_TRANSFER CHEQUE
description	TEXT	NULLABLE	Optional notes or reference number
receipt_url	VARCHAR(500)	NULLABLE	Relative path or full URL to uploaded receipt
employee_id	UUID	FK → employees.id, NULLABLE	Required when category=SALARY, else null
loan_payment_id	UUID	FK → loan_payments.id, NULLABLE	Set when category=LOAN_REPAYMENT
created_by	UUID	FK → users.id	User who recorded this expense
created_at	TIMESTAMP	DEFAULT NOW()	Auto-set on insert
updated_at	TIMESTAMP	AUTO UPDATE	Auto-updated on change

Expense Category ENUM values: MARINE_BILLS | SALARY | MAINTENANCE_REPAIR | FUEL_COST | FOOD_GROCERY | PORT_CASH | ZMA_BILLS | LOAN_REPAYMENT | ADMINISTRATION_COST | OTHER_COST

Indexes: COMPOSITE INDEX on (ship_id, date). COMPOSITE INDEX on (ship_id, category).

3.7 Table: loans

Column	Type	Constraint	Description
id	UUID	PK	Unique loan identifier
bank_name	VARCHAR(200)	NOT NULL	Lending institution name
total_amount	DECIMAL(16,2)	NOT NULL, CHECK(>0)	Original disbursed loan total
monthly_payment	DECIMAL(14,2)	NOT NULL, CHECK(>0)	Expected monthly installment amount

remaining_balance	DECIMAL(16,2)	NOT NULL	Current outstanding balance (decremented on each payment)
start_date	DATE	NOT NULL	Loan disbursement / agreement date
status	LoanStatus ENUM	DEFAULT ACTIVE	Values: ACTIVE PAID_OFF
description	TEXT	NULLABLE	Loan purpose or additional notes
created_by	UUID	FK → users.id	User who registered this loan
created_at	TIMESTAMP	DEFAULT NOW()	Auto-set on insert
updated_at	TIMESTAMP	AUTO UPDATE	Auto-updated on change

3.8 Table: loan_payments

Column	Type	Constraint	Description
id	UUID	PK	Unique payment record identifier
loan_id	UUID	FK → loans.id, NOT NULL	The loan this payment reduces
amount_paid	DECIMAL(14,2)	NOT NULL, CHECK(>0)	Amount paid in this installment
payment_date	DATE	NOT NULL	Date payment was made
notes	TEXT	NULLABLE	Bank reference, cheque number, etc.
created_by	UUID	FK → users.id	User who recorded this payment
created_at	TIMESTAMP	DEFAULT NOW()	Auto-set on insert

After every loan_payment insert, a Prisma \$transaction MUST: (1) decrement loans.remaining_balance by amount_paid, (2) if remaining_balance <= 0, set loans.status = PAID_OFF.

4. FILE-BY-FILE DOCUMENTATION

4.1 server.js — HTTP Entry Point

Property	Detail
Purpose	Start the HTTP server on the configured port
Responsibilities	Import Express app, read PORT from process.env, call app.listen(), log "Server running on port X"
Must NOT handle	Routes, middleware, business logic, database
Dependencies	src/app.js, process.env.PORT, dotenv
Variables	app (imported), PORT (number from env, default 5000)

Edge Cases	EADDRINUSE → log clear error. Unhandled promise rejections → log and exit process
------------	---

4.2 src/app.js — Express App Factory

Property	Detail
Purpose	Create, configure, and export the Express application
Responsibilities	Apply helmet, cors, express.json, express.urlencoded. Mount routes from src/routes/index.js. Mount error.middleware.js LAST.
Must NOT handle	Business logic, database calls, authentication
Middleware order	1.helmet() 2.cors({origin:FRONTEND_URL}) 3.express.json() 4.express.urlencoded() 5.routes 6.error handler
Route prefix	All routes mounted under /api/v1
CORS	Must whitelist process.env.FRONTEND_URL — wildcard (*) forbidden in production

4.3 src/routes/index.js — Route Aggregator

Module	Mount Path
auth.routes.js	/api/v1/auth
users.routes.js	/api/v1/users
ships.routes.js	/api/v1/ships
income.routes.js	/api/v1/income
expenses.routes.js	/api/v1/expenses
employees.routes.js	/api/v1/employees
loans.routes.js	/api/v1/loans
reports.routes.js	/api/v1/reports

4.4 src/config/database.js — Prisma Singleton

Property	Detail
Purpose	Export single shared PrismaClient instance — prevents connection pool exhaustion
Variable	prisma = new PrismaClient()
Must NOT handle	Query logic, transactions, business rules
Used by	ALL *.service.js files — only services touch the database

Test environment	Uses TEST_DATABASE_URL env var when NODE_ENV=test
------------------	---

4.5 src/config/constants.js — System Constants

All constants in UPPER_SNAKE_CASE. No imports from other app files. Zero side effects.

- USER_ROLES: Object — { ADMIN: "ADMIN", ACCOUNTANT: "ACCOUNTANT", MANAGER: "MANAGER" }
- SHIP_STATUS: Object — { ACTIVE: "ACTIVE", INACTIVE: "INACTIVE", MAINTENANCE: "MAINTENANCE" }
- INCOME_TYPES: Object — { SALES: "SALES", DELIVERY_ORDER: "DELIVERY_ORDER" }
- EXPENSE_CATEGORIES: Object — all 10 category values as key:value pairs
- PAYMENT_METHODS: Object — { CASH, BANK_TRANSFER, CHEQUE }
- LOAN_STATUS: Object — { ACTIVE, PAID_OFF }
- HTTP_STATUS: Object — { OK:200, CREATED:201, BAD_REQUEST:400, UNAUTHORIZED:401, FORBIDDEN:403, NOT_FOUND:404, CONFLICT:409, UNPROCESSABLE:422, SERVER_ERROR:500 }
- PAGINATION: Object — { DEFAULT_PAGE:1, DEFAULT_LIMIT:20, MAX_LIMIT:100 }

4.6 src/middlewares/auth.middleware.js

Property	Detail
Exported function	authenticate(req, res, next)
Input	Authorization header: "Bearer <jwt_token>"
Processing	Strips "Bearer ", calls jwt.verify(token, JWT_SECRET)
On success	Sets req.user = { id, email, role } — calls next()
Failure: missing header	Returns 401: "Authorization token required"
Failure: invalid token	Returns 401: "Invalid or malformed token"
Failure: expired token	Returns 401: "Token has expired"
Dependencies	jsonwebtoken, process.env.JWT_SECRET

4.7 src/middlewares/role.middleware.js

Property	Detail
Exported function	authorize(...allowedRoles) — factory returning middleware(req, res, next)
Input	req.user.role (set by authenticate middleware)
Processing	Checks if req.user.role is included in allowedRoles array
On success	Calls next()
Failure	Returns 403: "You do not have permission to perform this action"
Usage example	authorize(USER_ROLES.ADMIN, USER_ROLES.ACCOUNTANT)

4.8 src/middlewares/validate.middleware.js

Property	Detail
Exported function	validate(joiSchema) — factory returning middleware(req, res, next)
Input	req.body (for POST/PATCH) or req.query (for GET with filters)
Processing	Calls joiSchema.validate(req.body, { abortEarly: false, stripUnknown: true })
On success	Replaces req.body with sanitized value. Calls next().
Failure	Returns 422 with array of { field, message } error objects

4.9 src/middlewares/upload.middleware.js

Property	Detail
Exported	upload.single("receipt") — multer middleware for single file field named "receipt"
Storage	diskStorage: destination=./uploads, filename=Date.now() + original extension
File filter	Allowed: image/jpeg, image/png, application/pdf — rejects others with 400 error
Size limit	process.env.MAX_FILE_SIZE (default 5,242,880 bytes = 5MB)
Result	req.file.path contains the stored file path — saved to expense_records.receipt_url
Used by	expenses.routes.js on POST / and PATCH /:id routes only

4.10 src/middlewares/error.middleware.js

Property	Detail
Signature	errorHandler(err, req, res, next) — 4-argument Express error handler
Placement	MUST be the very last app.use() call in app.js
Prisma P2002	Unique constraint violation → 409 Conflict
Prisma P2025	Record not found → 404 Not Found
Custom errors	Uses err.statusCode if set, else defaults to 500
Development	Includes err.stack in response body for debugging
Production	Hides stack trace; returns generic "Internal server error" for 500s

4.11 src/utis/asyncHandler.js

Purpose: Eliminate try/catch boilerplate from every controller function.

```
const asyncHandler = (fn) => (req, res, next) =>
  Promise.resolve(fn(req, res, next)).catch(next);
```

All controller functions MUST be wrapped: `router.get("/", asyncHandler(controller.listShips))`

4.12 src/utlis/apiResponse.js

Function	Signature & Output
<code>sendSuccess()</code>	<code>sendSuccess(res, data, message="Success", statusCode=200) → { success:true, message, data }</code>
<code>sendError()</code>	<code>sendError(res, message, statusCode=500, errors=[]) → { success:false, message, errors }</code>

4.13 src/utlis/pagination.js

Function	Detail
<code>getPagination(query)</code>	Input: <code>req.query</code> . Reads page, limit. Enforces <code>MAX_LIMIT</code> . Returns <code>{ skip, take, page, limit }</code> .
<code>getPaginationMeta()</code>	Input: total (count), page, limit. Returns <code>{ total, page, limit, total_pages, has_next, has_prev }</code> .

4.14 src/utlis/dateHelpers.js

Function	Detail
<code>parseDateRange(query)</code>	Input: <code>{ start_date, end_date }</code> strings. Returns <code>{ gte: Date, lte: Date }</code> for Prisma where clause.
<code>getMonthBoundaries(year, month)</code>	Returns first day 00:00:00 and last day 23:59:59 of given month.
<code>formatDate(date)</code>	Returns ISO date string (YYYY-MM-DD) from a Date object.

4.15 Module Pattern: *.routes.js (All Modules)

Each routes file follows this strict pattern:

- Import: `express, authenticate, authorize, validate, asyncHandler, controller functions, validation schemas`
- Create: `const router = express.Router()`
- Define routes with full middleware chain:

```
router.post("/", authenticate, authorize(ADMIN, ACCOUNTANT),
  validate(createIncomeSchema), asyncHandler(incomeController.create))
```

- Export: `module.exports = router`
- MUST NOT contain any logic — only route wiring

4.16 Module Pattern: *.controller.js (All Modules)

- Import: `asyncHandler` (applied at routes level), `apiResponse utils`, `service module`

- Each function receives (req, res, next) — NO try/catch needed (asyncHandler catches)
- Extract data from req.body, req.params.id, req.query, req.user.id
- Call service function with extracted primitives (never pass req/res to service)
- Call sendSuccess() with service return value
- MUST NOT import prisma directly
- Function names: create, list, getById, update, remove (consistent across all modules)

4.17 Module Pattern: *.service.js (All Modules)

- Import: prisma from src/config/database.js, constants
- Each function is async and contains all database queries and business logic
- On not found: const err = new Error("Ship not found"); err.statusCode = 404; throw err;
- MUST NOT access req, res, or next
- Returns plain data objects — never HTTP responses
- Complex multi-step operations use prisma.\$transaction([...])

4.18 Module Pattern: *.validation.js (All Modules)

- Import: Joi
- Exports: one Joi schema object per endpoint requiring body validation
- Naming: createShipSchema, updateShipSchema, loginSchema, createIncomeSchema, etc.
- ENUM values imported from constants.js — never hardcoded strings
- All schemas use Joi.object({ ... })

5. API ENDPOINTS REFERENCE

5.1 Auth — /api/v1/auth

Method	Path	Auth	Description
POST	/auth/login	None	Authenticate with email+password, returns JWT token and user object
GET	/auth/me	JWT	Return current authenticated user profile from token payload

POST /auth/login — Full Specification

Property	Detail
Request Body	{ email: string (required, valid email), password: string (required, min 8 chars) }
Processing	Find user by email → bcrypt.compare(password, hash) → check is_active → sign JWT
Success 200	{ success:true, data:{ token:string, expiresIn:string, user:{ id, name, email, role } } }

Error 401	Email not found OR password mismatch — same error message to prevent user enumeration
Error 403	Account deactivated (is_active=false)

5.2 Users — /api/v1/users (ADMIN only)

Method	Path	Auth	Description
GET	/users	ADMIN	List all users. Query: role, is_active, page, limit
POST	/users	ADMIN	Create new user. Hashes password internally. Returns user without password_hash.
GET	/users/:id	ADMIN	Fetch single user by UUID
PATCH	/users/:id	ADMIN	Update name, role, or is_active flag
DELETE	/users/:id	ADMIN	Soft-delete: sets is_active=false. Never hard-deletes.

5.3 Ships — /api/v1/ships

Method	Path	Auth	Description
GET	/ships	All roles	List ships. Query: status, page, limit
POST	/ships	ADMIN	Create ship. Body: name, registration_number, status, description
GET	/ships/:id	All roles	Get ship + aggregated stats (total_income, total_expenses, net_profit, employee_count)
PATCH	/ships/:id	ADMIN	Update ship fields. Partial update (PATCH semantics).
DELETE	/ships/:id	ADMIN	Soft-delete (status=INACTIVE). Blocked if ship has income or expense records.

5.4 Income — /api/v1/income

Method	Path	Auth	Description
GET	/income	All roles	List income records. Query: ship_id, type, start_date, end_date, page, limit
POST	/income	ADMIN, ACCT	Record income. Body: ship_id, type, amount, date, description
GET	/income/:id	All roles	Get single income record by UUID
PATCH	/income/:id	ADMIN, ACCT	Update income record fields (partial)
DELETE	/income/:id	ADMIN	Hard delete income record. No soft delete.

POST /income — Validation Rules

Field	Validation Rules
ship_id	UUID format, required. Service verifies ship exists and status=ACTIVE.
type	ENUM: SALES DELIVERY_ORDER. Required.
amount	Positive number > 0. Max precision: 14 digits, 2 decimal places. Required.
date	ISO 8601 date string (YYYY-MM-DD). Required. Cannot be more than 1 year in the future.
description	String. Optional. Max 1000 characters.

5.5 Expenses — /api/v1/expenses

Method	Path	Auth	Description
GET	/expenses	All roles	List expenses. Query: ship_id, category, start_date, end_date, page, limit
POST	/expenses	ADMIN, ACCT	Record expense. multipart/form-data (receipt optional). See body rules below.
GET	/expenses/:id	All roles	Get single expense record with ship and employee details
PATCH	/expenses/:id	ADMIN, ACCT	Update expense fields. Partial update.
DELETE	/expenses/:id	ADMIN	Hard delete expense. Also deletes receipt file if present.

POST /expenses — Validation Rules

Field	Validation Rules
ship_id	UUID format, required. Verified to exist.
category	ENUM value from EXPENSE_CATEGORIES. Required.
amount	Positive number > 0. Required.
date	ISO date string. Required.
payment_method	ENUM: CASH BANK_TRANSFER CHEQUE. Required.
description	String, optional, max 1000 chars.
employee_id	Conditional: REQUIRED when category=SALARY. Must belong to same ship_id.
receipt (file)	Optional. multipart/form-data field "receipt". JPEG/PNG/PDF only. Max 5MB.

5.6 Employees — /api/v1/employees

Method	Path	Auth	Description
GET	/employees	All roles	List employees. Query: ship_id, is_active, page, limit
POST	/employees	ADMIN	Add employee and assign to ship. Body: ship_id, name, role, base_salary
GET	/employees/:id	All roles	Get employee details including ship info
PATCH	/employees/:id	ADMIN, ACCT	Update name, role, base_salary, ship_id (reassignment), is_active
DELETE	/employees/:id	ADMIN	Soft-delete: sets is_active=false. Does not cascade delete salary expenses.

5.7 Loans — /api/v1/loans

Method	Path	Auth	Description
GET	/loans	All roles	List loans. Query: status, page, limit
POST	/loans	ADMIN, ACCT	Register new loan. Sets remaining_balance = total_amount on creation.
GET	/loans/:id	All roles	Get loan details including full payment history array
POST	/loans/:id/payments	ADMIN, ACCT	Record payment. Atomically decrements remaining_balance. Auto-sets PAID_OFF.
GET	/loans/:id/payments	All roles	List all payments for a loan. Ordered by payment_date DESC.

5.8 Reports — /api/v1/reports

Method	Path	Auth	Description
GET	/reports/profit-loss	All roles	P&L report. Query: ship_id (optional), start_date, end_date (required)
GET	/reports/expenses	All roles	Expense breakdown by category. Query: ship_id, start_date, end_date
GET	/reports/income	All roles	Income breakdown by type. Query: ship_id, start_date, end_date
GET	/reports/loans	All roles	Loan summary: total_borrowed, total_paid, remaining_balance, active count
GET	/reports/dashboard	All roles	Dashboard KPIs: total income, expenses, profit, ship count, employee count

GET /reports/profit-loss — Response Structure

Field	Description
-------	-------------

total_income	SUM of income_records.amount for the date range (filtered by ship if ship_id provided)
total_expenses	SUM of expense_records.amount for the date range
net_profit	total_income – total_expenses (can be negative — represents loss)
by_ship	Array of { ship_id, ship_name, total_income, total_expenses, net_profit } for each ship
period	{ start_date: "YYYY-MM-DD", end_date: "YYYY-MM-DD" }
generated_at	ISO timestamp of when the report was calculated

6. SYSTEM REQUEST LIFECYCLE

6.1 Standard Request Pipeline

Step	What Happens
1. Client Request	HTTP request arrives at Express (server.js → app.js)
2. Security Headers	helmet() sets X-Content-Type-Options, HSTS, CSP, X-Frame-Options
3. CORS Check	cors() verifies Origin header against FRONTEND_URL
4. Body Parsing	express.json() + express.urlencoded() parses request body
5. Route Matching	src/routes/index.js forwards to correct module router
6. Authentication	authenticate() verifies JWT → sets req.user = {id, email, role}
7. Authorization	authorize(...roles) checks req.user.role against allowed roles
8. File Upload (if)	upload.single("receipt") processes multipart data → req.file
9. Validation	validate(schema) runs Joi → replaces req.body with sanitised data
10. Controller	Extracts data from req → calls service → calls sendSuccess()
11. Service	Executes Prisma queries → applies business rules → returns data or throws
12. Error Handler	Any throw anywhere → error.middleware.js → standardised JSON error
13. Response	{ success, message, data } JSON sent to client

6.2 Authentication Flow

- Client: POST /api/v1/auth/login with { email, password }
- auth.service.js: prisma.user.findUnique({ where: { email } })
- If null → throw 401: "Invalid credentials"
- bcrypt.compare(password, user.password_hash)
- If false → throw 401: "Invalid credentials" (same message — prevents email enumeration)

- If `user.is_active === false` → throw 403: "Account deactivated"
- `jwt.sign({ id: user.id, email: user.email, role: user.role }, JWT_SECRET, { expiresIn: JWT_EXPIRES_IN })`
- Return: `{ token, expiresIn, user: { id, name, email, role } }`
- Subsequent requests: Authorization: Bearer <token> in header

6.3 Loan Repayment Flow (Multi-Step Atomic Transaction)

- `POST /api/v1/loans/:id/payments`
- Middleware chain: `authenticate` → `authorize(ADMIN, ACCOUNTANT)` → `validate(recordPaymentSchema)`
- `loans.controller.js`: extract `{ amount_paid, payment_date, notes }` from `req.body`, `loan_id` from `req.params.id`
- `loans.service.js` → `recordPayment(loanId, paymentData, userId)`:
 - Step 1: `prisma.loan.findUnique({ where: { id: loanId } })` — throws 404 if not found
 - Step 2: if `loan.status === PAID_OFF` → throw 400: "Loan is already paid off"
 - Step 3: if `amount_paid > loan.remaining_balance` → throw 400: "Payment exceeds remaining balance"
 - Step 4: `prisma.$transaction([create loan_payment, update loan.remaining_balance, conditionally set PAID_OFF])`
- Return: updated loan record with new `remaining_balance` and payment list

6.4 Error Flow (All Paths)

- Services throw: `const err = new Error("message"); err.statusCode = 404; throw err;`
- `asyncHandler` catches promise rejections → calls `next(err)`
- `error.middleware.js` receives `err` with `statusCode` property
- Prisma error codes mapped: P2002→409, P2025→404, others→500
- Response always: `{ success: false, message: string, errors: [] }`

7. NAMING CONVENTIONS (STRICT)

These conventions are mandatory and must be applied consistently throughout the entire codebase.

7.1 File Naming

File Type	Convention — Example
Route files	kebab-case.routes.js — <code>ships.routes.js</code> , <code>loan-payments.routes.js</code>
Controller files	kebab-case.controller.js — <code>income.controller.js</code>
Service files	kebab-case.service.js — <code>expenses.service.js</code>
Validation files	kebab-case.validation.js — <code>employees.validation.js</code>
Middleware files	kebab-case.middleware.js — <code>auth.middleware.js</code>
Utility files	camelCase.js — <code>asyncHandler.js</code> , <code>dateHelpers.js</code>
Config files	camelCase.js — <code>database.js</code> , <code>constants.js</code>

7.2 JavaScript Variable & Function Naming

Context	Convention — Example
Local variables	camelCase — shipId, incomeRecord, totalAmount, remainingBalance
Constants (module)	UPPER_SNAKE_CASE — USER_ROLES, DEFAULT_LIMIT, JWT_SECRET
Service functions	camelCase verb+Noun — createShip, getAllIncomeRecords, recordLoanPayment, getProfitLoss
Controller functions	camelCase — create, list, getById, update, remove (consistent across all modules)
Middleware functions	camelCase descriptive — authenticate, authorize, validate, asyncHandler
Boolean variables	is/has prefix — isActive, hasReceipt, isPaidOff, isAdmin
Schema objects	camelCase + Schema suffix — createShipSchema, loginSchema, recordPaymentSchema

7.3 Database Naming

Element	Convention — Example
Table names	snake_case plural — users, ships, income_records, expense_records, loan_payments
Column names	snake_case — ship_id, created_at, payment_method, base_salary, receipt_url
Primary keys	Always named "id" — UUID type generated by database
Foreign keys	singular_referenced_table + _id — ship_id, loan_id, employee_id, created_by
ENUM values	UPPER_SNAKE_CASE — ACTIVE, LOAN_REPAYMENT, BANK_TRANSFER, DELIVERY_ORDER
Indexes	idx_tablename_columns — idx_income_records_ship_id_date
Prisma model names	PascalCase singular — User, Ship, IncomeRecord, LoanPayment

7.4 API Response Field Naming

Element	Convention
All response fields	snake_case — ship_id, total_income, created_at, remaining_balance
Pagination meta	Always: { total, page, limit, total_pages, has_next, has_prev }
List response	{ success:true, data:{ records:[], meta:{ pagination } } }
Single record	{ success:true, message:"", data:{ ...record } }
Error response	{ success:false, message:"", errors:[] }

8. ENVIRONMENT CONFIGURATION

8.1 All Environment Variables

Variable	Purpose & Example Value
NODE_ENV	Runtime mode. Values: development production test. Affects error verbosity.
PORT	HTTP server listen port. Example: 5000
DATABASE_URL	PostgreSQL connection string. Format: postgresql://USER:PASS@HOST:5432/ffms_db
JWT_SECRET	Secret for signing JWT tokens. MUST be minimum 64-character random string. Never commit to git.
JWT_EXPIRES_IN	Token expiry duration. Example: 7d or 24h
FRONTEND_URL	Allowed CORS origin. Example: https://fleet.company.com or http://localhost:3000 (dev)
MAX_FILE_SIZE	Max receipt upload size in bytes. Default: 5242880 (5MB)
UPLOAD_DIR	Local storage path for receipts. Example: ./uploads
BCRYPT_ROUNDS	Password hash salt rounds. Minimum 12 for production. Example: 12
TEST_DATABASE_URL	Separate database URL for test environment. Prevents test pollution.

8.2 Core npm Dependencies

Package	Purpose
express	HTTP server and routing framework
@prisma/client	Type-safe database query client (auto-generated from schema)
prisma	CLI: schema management, migrations, studio (devDependency)
jsonwebtoken	JWT creation (sign) and verification (verify)
bcryptjs	Password hashing — pure JavaScript, no native binary dependencies
cors	CORS policy enforcement middleware
helmet	Sets 11 security-related HTTP response headers
joi	Schema-based request validation library
multer	multipart/form-data parsing for file uploads
dotenv	Loads .env file into process.env
nodemon	Auto-restart on file changes in development (devDependency)

8.3 package.json Scripts

Script	Command & When to Use
start	node server.js — production server start
dev	nodemon server.js — development with auto-restart
db:migrate	prisma migrate dev -- name migration_name — apply schema changes with migration history
db:push	prisma db push — apply schema directly (development only, no migration file)
db:generate	prisma generate — regenerate Prisma client after schema changes
db:studio	prisma studio — open database GUI at localhost:5555
db:seed	node prisma/seed.js — seed initial admin user and sample ships

9. INTER-FILE RELATIONSHIP MAP

9.1 Complete Dependency Chain: Creating an Income Record

Layer	File
1. Entry	server.js imports and starts src/app.js
2. App	src/app.js mounts src/routes/index.js
3. Router	src/routes/index.js mounts income.routes.js at /api/v1/income
4. Auth Middleware	src/middlewares/auth.middleware.js — authenticate()
5. Role Middleware	src/middlewares/role.middleware.js — authorize(ADMIN, ACCOUNTANT)
6. Validation	src/middlewares/validate.middleware.js + src/modules/income/income.validation.js
7. Controller	src/modules/income/income.controller.js — create()
8. Service	src/modules/income/income.service.js — createIncomeRecord()
9. Database	src/config/database.js — prisma.incomeRecord.create()
10. Constants	src/config/constants.js — INCOME_TYPES.SALES for ENUM validation
11. Response	src/utils/apiResponse.js — sendSuccess()
12. Error path	src/middlewares/error.middleware.js — if anything throws

9.2 Complete Dependency Chain: Loan Payment Recording

Layer	File
1-5. Same as above	server → app → routes → authenticate → authorize(ADMIN, ACCOUNTANT)
6. Validation	src/modules/loans/loans.validation.js — recordPaymentSchema
7. Controller	src/modules/loans/loans.controller.js — recordPayment()
8. Service	src/modules/loans/loans.service.js — recordLoanPayment()
9. Prisma \$transaction	(1) prisma.loanPayment.create() (2) prisma.loan.update({remaining_balance})
10. Conditional	If remaining_balance <= 0: prisma.loan.update({ status: PAID_OFF })
11. Response	sendSuccess() with updated loan object

9.3 All Files That Import prisma (database.js)

- src/modules/auth/auth.service.js
- src/modules/users/users.service.js
- src/modules/ships/ships.service.js
- src/modules/income/income.service.js
- src/modules/expenses/expenses.service.js
- src/modules/employees/employees.service.js
- src/modules/loans/loans.service.js
- src/modules/reports/reports.service.js

RULE: ONLY *.service.js files import prisma. Controllers, routes, and middlewares MUST NOT query the database directly.

9.4 All Files That Import constants.js

- All 8 *.validation.js files — use ENUM values in Joi schemas
- All 8 *.service.js files — use ENUM comparisons and default values
- src/middlewares/auth.middleware.js — uses HTTP_STATUS
- src/middlewares/role.middleware.js — uses USER_ROLES, HTTP_STATUS
- src/utils/apiResponse.js — uses HTTP_STATUS
- src/utils/pagination.js — uses PAGINATION.DEFAULT_LIMIT, PAGINATION.MAX_LIMIT

10. SECURITY GUIDELINES

10.1 Authentication Security

- Passwords hashed with bcryptjs — minimum 12 salt rounds (BCRYPT_ROUNDS env var)
- JWT signed with HS256 using JWT_SECRET — minimum 64-character random string
- Token expiry enforced — expired tokens rejected with 401
- Login error messages are identical for "email not found" and "wrong password" — prevents user enumeration

- `is_active` flag checked at login — deactivated users cannot obtain new tokens

10.2 Input Security

- All POST/PATCH bodies validated via Joi schemas before reaching services
- Prisma uses parameterized queries by default — SQL injection is not possible through normal ORM usage
- File uploads: MIME type validated server-side (multer fileFilter), not just by client-reported Content-Type
- Receipt filenames: stored with timestamp prefix, never original client filename (prevents path traversal)
- Numeric amounts: validated positive, reasonable precision limits

10.3 Access Control Rules

Route Category	Minimum Required Role
POST/PATCH/DELETE users	ADMIN only
POST/PATCH/DELETE ships	ADMIN only
POST/PATCH/DELETE employees	ADMIN (create/delete), ADMIN+ACCOUNTANT (update)
POST/PATCH income records	ADMIN + ACCOUNTANT
DELETE income records	ADMIN only
POST/PATCH expense records	ADMIN + ACCOUNTANT
DELETE expense records	ADMIN only
POST loans and payments	ADMIN + ACCOUNTANT
GET all read endpoints	All roles (ADMIN, ACCOUNTANT, MANAGER)
POST /auth/login	No authentication required

10.4 HTTP Security Headers (Helmet)

- X-Content-Type-Options: nosniff — prevents MIME type sniffing
- X-Frame-Options: DENY — prevents clickjacking
- Strict-Transport-Security: enabled in production — forces HTTPS
- Content-Security-Policy: configured to restrict script/style sources
- X-XSS-Protection: 1; mode=block — legacy XSS filter

11. PRISMA SCHEMA OVERVIEW

The complete prisma/schema.prisma file structure. This MUST match the database tables described in Section 3.

11.1 Generator & Datasource Block

```
generator client {
  provider = "prisma-client-js"
}
```

```
datasource db {
  provider = "postgresql"
  url      = env("DATABASE_URL")
}
```

11.2 ENUMs (Must Match constants.js Exactly)

```
enum UserRole      { ADMIN ACCOUNTANT MANAGER }
enum ShipStatus    { ACTIVE INACTIVE MAINTENANCE }
enum IncomeType    { SALES DELIVERY_ORDER }
enum PaymentMethod { CASH BANK_TRANSFER CHEQUE }
enum LoanStatus    { ACTIVE PAID_OFF }
enum ExpenseCategory {
  MARINE_BILLS SALARY MAINTENANCE_REPAIR FUEL_COST
  FOOD_GROCERY PORT_CASH ZMA_BILLS LOAN_REPAYMENT
  ADMINISTRATION_COST OTHER_COST
}
```

11.3 Prisma Model Name Conventions

Database Table	Prisma Model Name (PascalCase singular)
users	User
ships	Ship
employees	Employee
income_records	IncomeRecord
expense_records	ExpenseRecord
loans	Loan
loan_payments	LoanPayment

11.4 Critical Business Logic in Services (Summary)

- SALARY expense creation: services/expenses.service.js MUST verify employee.ship_id === expense.ship_id before insert
- Loan payment recording: MUST use prisma.\$transaction to atomically create payment AND update loan balance
- Remaining balance → 0: MUST auto-update loan.status = PAID_OFF in same transaction
- Ship deletion guard: ships.service.js MUST check income_records and expense_records count > 0 before allowing deletion
- All monetary aggregations in reports: use Prisma groupBy() or \$queryRaw with SUM() — never accumulate in JS loops
- Decimal precision: Prisma Decimal type used for all monetary fields — no JavaScript floating-point arithmetic on amounts

12. FUTURE IMPROVEMENTS

These are planned for future versions. Do NOT implement any of these in v1.0.

- Refresh Token System — httpOnly cookie-based refresh endpoint (/auth/refresh)
- Audit Log Table — record every data mutation with user, timestamp, old value, new value
- Notification Module — loan due date reminders, high-expense threshold alerts
- PDF/Excel Export — downloadable versions of all reports
- Banking API Integration — auto-import transactions from bank feeds
- Advanced Analytics — trend graphs, expense forecasting, seasonal patterns
- Multi-currency Support — currency field on all financial records with exchange rate conversion
- Mobile App API — dedicated endpoints optimised for mobile payloads
- Two-Factor Authentication — TOTP-based 2FA for ADMIN and ACCOUNTANT roles

END OF DOCUMENTATION

Fleet Financial Management System — Backend Blueprint v1.0.0

Prepared by Issa Ahmed — Zanzibar, Tanzania